Machine Learning System Design interview
MY CURRICULUM
01 Introduction and Overview
02 Feature Selection
03 Data Preparation
04 Google Street View Blurring System
05 Ad Click Prediction System
06 Video Recommendation System
07 Event Recommendation System
08 NSFW Production Text-to-Image Models
09 Recommendation News Feed
10 People You May Know

06
Video Recommendation System

Recommendation systems play a key role in video and music streaming services. For example, YouTube recommends videos to users as they watch recommended videos, and Spotify recommends songs and playlists to users.

In this chapter, we design a video recommendation system similar to YouTube's [1]. The system recommends videos on the user's homepage based on their profile, previous interactions, etc.

Figure 6.1: Homepage video recommendation

Recommendation systems are often very complex to design, and a good amount of engineering effort is required to develop an efficient and scalable system. Don't worry, though, no one expects you to build the perfect system in a 45 -minute interview. The goal of this chapter is to demonstrate your ability to ask clarifying questions, understand trade-offs, design a feasible system, and discuss ways to improve the system.

Clarifying Requirements

Here is a typical interaction between a candidate and an interviewer.

Candidate: Do we recommend videos to users on the homepage? Similar recommendation system to increase user engagement?
Interviewer: That's correct.

Candidate: Can we recommend another video to a video a user is watching right now? Or does it show a personalized list of videos on the user's homepage?
Interviewer: Let's focus on the homepage recommendation system, which recommends personalized videos to users when they load the homepage.

Candidate: Does YouTube have a global service, can I assume users worldwide and videos are in different languages?
Interviewer: That is a fair assumption.

Candidate: Can revenue be our construct the dataset based on user interactions with video content?
Interviewer: Yes, that is a good construct.

Candidate: Are a user group's videos together by category? Platform, such as the recommendation for the mobile during the morning commute?
Interviewer: For the sake of simplicity, let's assume the playlist features does not exist.

Candidate: How many videos are available on the platform?
Interviewer: Billions of videos on the platform.

Candidate: Has the system collected enough interactions to train a ML model? Can I assume the recommendation should not take many cold start videos?
Interviewer: That sounds good.

Let's summarize the problem statement. We are asked to design a homepage video recommendation system. The business objective is to increase user engagement. The system should rank billions of videos on the platform based on user interactions, such as video watches, likes, and dislikes. Users and videos can be in different languages. There are approximately 10 billion videos on the platform, and recommendations should be served quickly.

Frame the Problem as an ML Task

Defining the ML objective

The business objective of the system is to increase user engagement. There are several options available for translating business objectives into ML objectives. We will provide some of these and discuss their pros and cons.

Maximize the number of user clicks. A video recommendation system can be designed to maximize user clicks. However, this may result in clickbait videos. For example, a user may click on a cooking video but then quickly leave the video after realizing the thumbnail image has misleading. Not the video's content has the cooking. Instead, we want to recommend videos users watch and enjoy. This objective does not capture user satisfaction and engagement well.

Maximize watch time of videos recommended. The system could also recommend clickbait videos and likely watch to completion. A longer problem with this objective is that the model may recommend longer videos.

Maximize total watch time. This objective produces recommendations that users spend more time watching.

Maximize the number of relevant videos. This objective measures the number of videos that are relevant to users. Engagement on product changes can define relevance based on some criteria. Such criteria can be based on Facebook and Instagram user reactions. For example, Facebook uses "Like", "Love", "Haha", "Wow", "Sad", and "Angry" reactions to measure user satisfaction [2]. Using one define relevance, we can construct a dataset and use a model to predict the relevance score between a user and a video.

In this system, we choose the final objective as the ML objective because we have more control over what signals to use in addition. It does not have the shortcomings of the other options described earlier.

Specifying the system's input and output

As Figure 6.2 shows, a video recommendation system takes a user as input and outputs a ranked list of videos sorted by their relevance scores.

Figure 6.2: A video recommendation system's input-output

Choosing the right ML category

In this section, we examine three types of personalized recommendation systems:

- Content-based filtering
- Collaborative filtering
- Hybrid filtering

Figure 6.3: Content-based filtering

Here is an explanation of the diagram:

- User A engaged with videos 2 and 4 in the past.
- User B engaged with video 1 in the past.
- The system recommends videos 2 to user A

Content-based filtering has pros and cons.

Pros:

- **Ability to recommend new videos.** Unlike the collaborative filtering method, content-based filtering can recommend new videos to users even if they have not interacted with them.
- **Ability to capture the unique interests of users.** This is because we recommend videos based on users' previous engagements.

Cons:

- **Difficult to determine relevancy.** Unlike the collaborative filtering method, it is more difficult to determine how relevant a video is to a user.
- **Tends to discover users' new areas of interest.** The system can recommend videos about new topics that other users have engaged with in the past.
- **Cold start problem.** Cold start problem for new users.

Collaborative filtering has pros and cons.

Pros:

- **No domain knowledge needed.** It does not rely on video features, which means no domain knowledge is needed to recommend videos to users.
- **Easy to discover users' new areas of interest.** The system can recommend videos about new topics that other users have engaged with in the past.
- **Cold start problem.** Cold start problem for all users.

Cons:

- **Cold start problem.** This refers to a situation where there is insufficient data to recommend videos to users. For example, a new user or a new video. Cold start problem is a common issue in the field of recommendation systems. We will discuss some of the ways to address this issue in the next section. Cold start problem is not a problem for content-based filtering.
- **Difficult to capture the unique interests of users.** It is difficult for CF to handle users with specialized or niche interests. CF relies upon similar users to make recommendations, and it might be difficult to find users with niche interests.

Hybrid filtering combines both CF and content-based filtering. As Figure 6.4 shows, hybrid filtering combines CF trained and content-based recommendation separately, or it applies a single model, using both user-video and video-video features.

Figure 6.4: Hybrid filtering

Let's explore three approaches.

Video-level

- User-video pairs, where the video feature and their associated interactions, such as video ID, video length, video title, etc. Some of these features are categorical features that need to be one-hot encoded or embedded before they can be fed into the model. Such as the video ID.

Video ID	Length	Upload type	Title	Video ID	Interactions	Video	Language
1	120	User Video	Dog Training	1	1	Dog Training	EN
2	240	TV	Mars Landing	User ID	2	Mars Landing	Spanish
3	5600	Live Stream	Honeymoon in Bali	2	3	Honeymoon in Bali	Arabic

Figure 6.5: Video features

Users

The following artwork contents represents user data.

ID	Username	Age	Gender	City	Country	Language	Time zone
1	Tom	18	M	Paris	FR	FR	Europe/Paris
2	Jack	24	M	New York	US	EN	America/New_York
3	Mary	30	F	Los Angeles	US	EN	America/Los_Angeles
4	Ali	28	M	Dubai	AE	AR	Asia/Dubai

Figure 6.6: User profiles

User video interactions

The user video interaction table consists of various user interactions with the videos, including likes, clicks, interactions, and watch minutes, and includes timestamps to indicate when the interactions happened. Some of these features are numerical features. The following shows four user video interactions.

User ID	Video ID	Interaction type	Watch time	Location	Device	Timestamp
1	10	Like		US-SFO-37.7749,-122.4194	Mobile	1630451200
2	10	Impression		US-NYC-40.7128,-74.0060	Desktop	1630451200
2	8	Watch	45 minutes	US-NYC-40.7128,-74.0060	Desktop	1630452600
3	6	Click		US-LAX-34.0522,-118.2437	Mobile	1630453200
4	5	Search	Search keyword: "cooking"	US-CHI-41.8781,-87.6298	Desktop	1630455000
4	6	Comment	Amazing video. Thanks!	US-CHI-41.8781,-87.6298	Desktop	1630455007

Table 6.4: User video interaction data

Feature engineering

The ML system is expected to predict videos that are relevant to users. Let's engineer features to help the system make informed predictions.

Video features

Some important video features include:

- Video ID
- Duration
- Language
- Title and tags

Video ID

The ID, or categorical data, is represented them by numerical vectors, we use an embedding layer and the embedding layer is learned during model training.

Duration

This attribute approximately how long the video lasts from start to finish. This information is important since some users may prefer shorter videos, while others prefer longer videos.

Language

The language used in a video is an important factor. This is because users naturally prefer particular languages. Since language is a categorical variable and plays a little part of attribute values, we can an embedding layer to represent it.

Title and tags

Titles and tags are text features. They are either provided manually by the uploader or are explicitly predicted by computers, for example, the model can tag a video as sports if it sees sets equation predictions. Typically, a video title's Tokens or tags can indicates the video is related to pizza and cooking.

How to prepare it? We can use a pre-trained language trained model, such as DERT [3], to map them into feature series.

For the title, we map it into a feature vector using a context aware word embedding model, such as pre-trained BERT [4].

Figure 6.7 shows an overview of video features preparation.

Figure 6.7: Video feature preparation

User features

We categorize user features into the following buckets:

- User demographics
- Contextual information
- User interaction features

User demographics

An overview of user demographic features is shown in Figure 6.8.

Figure 6.8: Features related to user's demographics

Contextual information

Here are a few important features for capturing contextual information:

- **Time of day.** A user may watch videos at different times of day. For example, a software engineer may watch more educational videos during the evening.
- **Device.** Users may watch videos on different devices.
- **Day of the week.** Depending on the day of the week, users may have different preferences for videos.

Figure 6.9: Features related to contextual information

User historical interactions

User historical interactions play an important role in understanding user interests. A few features related to historical interactions include:

- Search Queries
- Liked videos
- Watched videos and impressions

Search History

Why is it important?

Previous searches indicate what the user looked for in the past, and past behavior is often an indicator of future behavior.

How to prepare it?

Use a pre-trained sentence embedding model, such as BERT, to map each search query into an embedding vector. Note that a user may have issued multiple searches in the past. To create a fixed-size feature vector, summarizing all the search queries, we compute the mean of embeddings.

Liked video

Why is it important?

This indicates a user finds particular videos to be helpful in determining which type of content they're interested in.

How to prepare it?

Video IDs are mapped into embedding vectors using the embedding layer. Similarly to search history, we average liked embeddings to get a fixed-size vector of liked videos.

Watched video and impressions

Why is it important?

The historical watched videos and "impressions" (or "recommendations" is very similar to what we did for liked videos), So we don't repeat it.

Figure 6.10 summarizes features related to user video interactions.

Figure 6.10: Features related to user video interactions

Model Development

In this section, we examine three recommendation system models that are typically employed in CF-based or content-based recommendations.

- Matrix factorization
- Two tower neural network

Matrix factorization

Collaborative filtering is the matrix factorization model. It is important to know what a feedback matrix is.

Feedback matrix

Also called a utility matrix, this is a matrix that represents users' opinions about videos. Figure 6.11 shows a sheet, user-video feedback matrix where each row represents a user, and each column represents a video. The entry in the matrix specify the user's opinion to it an "observed" or "positive".

Figure 6.11: User-video feedback matrix

How can we determine whether a user finds a recommended video relevant? We have three options:

- Explicit feedback
- Implicit feedback
- Combination of explicit and implicit feedback

Explicit feedback. In feedback matrix, a kind based on interactions that explicitly indicates a user's opinion about a video, such as "Like" or "Dislike". For example, YouTube allows users to express their opinions on videos by clicking the thumbs-up or thumbs-down button; however, few users engage in this feedback. The matrix is sparse since only a small fraction of users provide explicit feedback. Because sparsity matrix, models difficult to learn.

Implicit feedback. Implicit feedback refers to interactions that indirectly indicates a user's opinion about a video, such as "Watch" or "Watch" with watched minutes, video impressions, and clicks, resulting in a denser feedback matrix compared to explicit feedback. It does not directly reflect users' opinions and might be noisy.

Combination of explicit and implicit feedback. This option combines explicit and implicit feedback using heuristics.

What is the best option for building our feedback matrix?

Since the model needs to learn the values of the feedback matrix, it's important to build the matrix that aligns well with the ML objective as close earlier.

In this case, the ML objective is to maximize relevancy, where relevancy is defined as the combination of explicit and implicit feedback. As such, the best option of constructing explicit and implicit feedback is the last choice.

Matrix factorization model

Matrix factorization is a simple embedding model. The algorithm decomposes the user-video feedback matrix into the product of two lower-dimensional matrices, one for users and one for videos, so that each row represents a user embedding and each row represents a video embedding. In collaborating filtering, only the model needs to map each user into an embedding space and each video into an embedding space, such that their distance represents their relevance. Figure 6.12 shows that matrix factorization is decomposed into user and video embeddings.

Figure 6.12: User-video feedback matrix

1
1
1

Matrix factorization training

As part of training, we start by random user and video embedding matrices so that their product is a good approximation of feedback matrix (Figure 6.5):

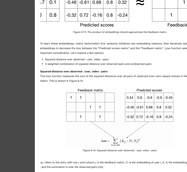

Figure 6.5: The product of embedding matrices should approximate the feedback matrix

To learn these embeddings, metric factorization first randomly initializes two embedding matrices, then iterates optimizes the embeddings to minimize the difference between the "feedback matrix" and "predicted scores" (user-feature vectors in the original embeddings). Loss function calculates this difference:

- Squared distance over observed pairs: $(1 - 0.6)^2$
- Squared distance over unobserved pairs: $(-1 - 0.9)^2$
- A weighted combination of squared distances over observed and unobserved pairs

Squared distance over observed pairs: $(\text{user}_i \cdot \text{video}_j - \text{pairs})^2$

Minimizing this term ensures that the predicted scores for observed interactions over all pairs of observed (non-zero values) entries in the feedback matrix. This is always in Figure 6.6:

Figure 6.6: Squared distance over all - user, video - pairs

Squared distance over unobserved pairs: $(\text{user}_i \cdot \text{video}_j - \text{pairs})^2$

Minimizing this term ensures that predicted scores for unobserved interactions are lower than observed interactions. However, this loss has a trivial solution: if all embeddings are zero, the product will be zero, and the model will predict zero for all user-feature combinations, regardless of ranking. This results in predictions that are barely close to zero. This is not useful and leads to poor generalization performance on unseen - user, video - pairs.

A weighted combination of squared distances over observed and unobserved pairs

To overcome the shortcomings of the two functions described below, use a fixed-weight combination of both:

Figure 6.7: Weighted combination of both - user, video - pairs

The first summation is the loss formula calculated for the one or the observed pairs, and the second summation calculates the loss calculated over unobserved pairs. It is a hyperparameter that weights the two summations. It chooses more and decreases the other in the training phase. The intuition behind this is that most of cases will in practice, the model will choose the loss function for the unobserved pairs.

Matrix factorization optimization

So far we only mentioned an optimization algorithm is required. Two commonly used optimization algorithms in metric factorization are:

- Stochastic Gradient Descent (SGD). This optimization algorithm is used to minimize losses (6).
- Weighted Alternating Least Squares (WALS). This optimization algorithm is used to factorize factorization. The process is little bit more complicated than SGD.

WALS is used for factorizing sparse (i.e. and optimize the other embedding (4)

1. Fix the other embedding (users for), and optimize the embedding vectors (v)
2. Repeat

WALS usually converges faster and is parallelizable. To learn more about WALS, see (WALS), Here we use WALS because it converges faster.

Matrix factorization inference

To predict the relevance between an arbitrary user and a candidate video, we calculate the similarity between the user embedding and the candidate video, such as a dot product. For example, as shown in Figure 6.8, the relevance score between user 2 and video 4 is 0.65:

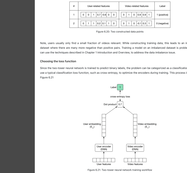

Figure 6.8: shows the predicted scores for all the - user, video - pairs. The system recommends recommended videos based on relevance scores.

Figure 6.9: Predicted personal relevance scores

Reminder:

Since embeddings are random even user-video interactions only, it is commonly used in collaborative filtering.

Before wrapping up metric factorization, let's discuss the pros and cons of the model.

Pros:

- Training speed: Metric factorization is efficient during the training phase. This is because there are only two embedding matrices to learn.
- Inference speed: Metric factorization is fast at serving time. The learned embeddings are static, meaning that once we learn them, we can save them and use them to compute the dot product at serve time.

Cons:

- Metric factorization only relies on user-video interactions, it does not use other features, such as the user's age or language. This limits the prediction capability of the model because features like language are useful to improve the quality of recommendations.
- Handling cold users is difficult. At serve time, there are not enough interactions for the model to produce meaningful embeddings. Therefore, metric factorization cannot determine whether a video is relevant to user by computing the dot product between their embeddings.

Let's see how two-tower neural networks address the shortcomings of metric factorization.

Two-tower neural network

A two-tower neural network composes two encoder towers: the user tower and the video tower. The user encoder takes user features as input and maps them to user embeddings. The video tower does the same thing, but it maps video features to video embeddings. Similarly to metric factorization, the relevance between two embeddings is the dot product between two embeddings.

Figure 6.10 shows the two-tower architecture. In contrast to metric factorization, neural architectures are flexible enough to incorporate all kinds of features to better capture the user's specific interests.

Figure 6.10: Two-tower neural network

Constructing the dataset

Two-tower models are supervised learning features from different - user, video - pairs and labeling them as positive or negative based on the user's interaction with the video. For example, we can label the pair of user watches and finishes a video as a positive example.

To construct negative data points, we can either choose random videos and we can relevant or choose those the user explicitly dislikes by pressing the thumbs down button. Figure 6.11 shows an example of the training data.

Figure 6.11: Two-connected datapoints

Note, users usually only find a small fraction of videos relevant. While constructing training data, this leads to an imbalanced dataset with many more negative than positive examples. Therefore, we need to adjust the loss function to account for imbalance. Let's see two techniques described in Chapter 1 Introduction and Scoring, to address the class imbalance issue.

Choosing the loss function

Since the two-tower neural network is similar to matrix factorization, we can adapt the loss function we used for matrix factorization. We can use the cross-entropy loss function, such as cross-entropy, to optimize the embedding learning. The process is similar to Figure 6.12:

Figure 6.12: Two-tower neural network trained by collaborative filtering

Let's see the pros and cons of a two-tower neural network model.

Pros:

- **Utilizes user features:** The model accepts user features, such as age and gender, as input. These preference features help the model learn user interest more accurately.
- **Handles cold users:** The model easily handles new users at inference time. For example, if a user is new, age, gender, etc.

Cons:

- **Slower serving:** The model needs to compute the user embedding at serve time. This makes the model slower to serve. However, in practice, it can be fast enough for content based filtering. The model needs to transform video features into video embeddings for each candidate video at inference time.
- **Cold start problem:** The model still struggles with new videos. The model learns how to recommend videos from user's interactions. Therefore, if the training is not adequate, it may recommend irrelevant videos.

Metric factorization vs. two-tower neural network

Table 6.2 summarizes the differences between metric factorization and two-tower neural network architecture.

	Metric factorization	Two-tower neural network
Training cost	• Metric efficient to train	• More costly to train
Inference speed	• Faster as embeddings are static and precomputed	• Slower as embeddings are computed at query time
Cold start problem	• Cannot handle new users easily	• Handles users a bit better on user features
Quality of recommendations	• Not ideal since the model does not use contextual features	• Better recommendations since it relies on more features

Table 6.2: Metric factorization vs. two-tower neural networks

Evaluation

The content-ranking framework can be calculated with offline and online metrics.

Offline metrics

We evaluate the following offline metrics commonly used in recommendation systems.

Precision@k. This metric measures the proportion of relevant videos among the top k recommended videos. Multiple k values (e.g., 1, 5, 10) are used.

mAP. This metric measures the ranking quality of recommended videos. It is a good fit because the relevance scores are binary (1 for relevant, 0 for irrelevant).

Diversity. This metric measures how diverse recommended videos are by each user. This metric is important to users, as users usually prefer a diverse set of videos. For example, if a user watches many videos about cooking, other relevant videos about cooking or other popular content relevant to the user may provide better diversity and better user experience.

Note that using diversity as the only measure of quality can result in misleading recommendations. For example, if the recommended videos are all popular videos, the diversity score is high, but the recommendations lacking personalization. Therefore, we should always utilize other offline metrics to ensure both relevance and diversity.

Online metrics

In practice, companies track many metrics during online evaluation. Let's examine some of the most important ones.

- Click-through rate (CTR)
- The minutes of watch time
- Total watch time
- Explicit user feedback

CTR. This ratio between clicked videos and the total number of recommended videos. The formula is:

$$\text{CTR} = \frac{\text{number of clicked videos}}{\text{number of recommended videos}}$$

CTR is an ingredient metric to track user engagement, but the drawback of CTR is that we cannot capture or measure clickbait videos.

The number of completed videos. This number of recommended videos that users watch until the end. By tracking this metric, we can underestimate the effect the recommender makes from users' content.

Total watch time. This time that users spend watching the recommended videos. Which recommendations interest users, they spend more time watching videos, overall.

Explicit user feedback. The total number of videos that users explicitly liked or disliked. The metric accounts letters users' opinions of recommendations.

Serving

At serving time, the system recommends the most relevant videos to a given user by ranking the selection chain from different stages. In this section, we will propose a production pipeline that is both efficient and accurate at serving recommendations.

Let's define the term "candidate" to refer to a video that is being ranked in the pipeline. Let's define the term "slate" to refer to a list of candidates that the system will recommend to a user. Let's define the term "impression" to refer to a candidate that the system has recommended to a user (for example, it is a non-stage. Another, it is a non-stage. Another, it is a non-stage. Another

2. DNN for YouTube recommendation. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf.
3. CBOW paper. https://arxiv.org/pdf/1301.3781.pdf.
4. BERT paper. https://arxiv.org/pdf/1810.04805.pdf.
5. Matrix factorization. https://developers.google.com/machine-learning/recommendation/collaborative/matrix.
6. Stochastic gradient descent. https://en.wikipedia.org/wiki/Stochastic_gradient_descent.
7. WALS optimization. https://fairyonice.github.io/Learn-about-collaborative-filtering-and-weighted-alternating-least-square-with-tensorflow.html.
8. Instagram multi-stage recommendation system. https://ai.facebook.com/blog/powered-by-ai-instagrams-explore-recommender-system/.
9. Exploration and exploitation trade-offs. https://en.wikipedia.org/wiki/Multi-armed_bandit.
10. Bias in AI and recommendation systems. https://www.searchenginejournal.com/biases-search-recommender-systems/339319/#close.
11. Ethical concerns in recommendation systems. https://link.springer.com/article/10.1007/s00146-020-00950-y.
12. Seasonality in recommendation systems. https://www.computer.org/csdl/proceedings-article/big-data/2019/09005954/1hJsfgT0qL6.
13. A multitask ranking system. https://daiwk.github.io/assets/youtube-multitask.pdf.
14. Benefit from a negative feedback. https://arxiv.org/abs/1607.04228?context=cs.

Mark as Complete

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.

Ask Alex